ĐẶC TẢ THIẾT KẾ gemm_accel_top

Mục lục

1 Tổng quan **3**
1.1 Phạm vi chức năng . . . 3
1.2 Quy ước . . . 3
1.3 Module . . . 3
1.4 Tham số hệ thống . . . 3
1.5 Tín hiệu giao tiếp . . . 4

2 Đặc tả thanh ghi **7**
2.1 Tổng hợp thanh ghi . . . 7
2.2 Thanh ghi điều khiển (CTRL_REG) . . . 8
2.3 Thanh ghi trạng thái (STATUS_REG) . . . 8
2.4 Kích thước ma trận (MATRIX_DIM) . . . 9
2.5 Địa chỉ byte của A (BASE_ADDR_IN) . . . 9
2.6 Địa chỉ byte của B (BASE_ADDR_WEIGHT) . . . 9
2.7 Địa chỉ byte vùng kết quả C hoặc Y (BASE_ADDR_OUT) . . . 10
2.8 Điều khiển hậu xử lý (POST_CTRL) . . . 10
2.9 Địa chỉ byte vector bias; bỏ qua khi POST_EN = 0 (BIAS_ADDR) . . . 11
2.10 Hệ số lượng tử (QUANT_M0) . . . 11
2.11 Lượng dịch lượng tử (QUANT_SHIFT) . . . 11
2.12 Cấu hình phần cứng (HW_CONFIG) . . . 12
2.13 Tổng số chu kỳ BUSY của tác vụ (PERF_TOTAL) . . . 12
2.14 Số chu kỳ LOAD, không tính FETCH (PERF_LOAD) . . . 13
2.15 Số chu kỳ COMPUTE (PERF_COMPUTE) . . . 13
2.16 Số chu kỳ STORE_C (PERF_STORE) . . . 13
2.17 Con trỏ descriptor đầu (DESC_PTR) . . . 14
2.18 Địa chỉ descriptor lỗi (ERR_DESC_ADDR) . . . 14
2.19 Số descriptor hoàn tất (DESC_DONE_CNT) . . . 15
2.20 Mã lỗi . . . 15
2.21 Yêu cầu thanh ghi . . . 15
2.22 Ngữ nghĩa tác vụ . . . 16
2.23 Quy tắc cho firmware . . . 16

3 Mô tả chức năng và dữ liệu **17**
3.1 Phép toán . . . 17
3.2 Kích thước ma trận . . . 18
3.3 Bố trí dữ liệu trong DRAM . . . 18
3.4 Mô hình tham chiếu . . . 18

4 Luồng vận hành **19**

4.1 Các pha của một tác vụ 19
4.2 Luồng một tác vụ 19
4.3 Quyền truy cập scratchpad theo pha 19
4.4 Xử lý lỗi 20
4.5 Reset và soft reset 20

5 Thành phần bổ sung **20**
5.1 Nguyên tắc chung 20
5.2 DMA theo chuỗi descriptor 21
5.3 Scratchpad ping-pong 22

6 Tích hợp các module **22**

1. Tổng quan

1.1. Phạm vi chức năng

- Mỗi tác vụ tính một phép GEMM: A là M×K, B là K×N, kiểu INT8 có dấu; tích lũy INT32; M $\leq$ 32, N $\leq$ 32, K $\leq$ 32.
- Kết quả ra DRAM ở một trong hai dạng: C (INT32, không hậu xử lý) hoặc Y (INT8, sau bias, requantize, ReLU tùy chọn, kẹp).
- CPU cấu hình qua AXI4-Lite (MMIO) và chờ kết quả bằng polling; DMA đọc/ghi DRAM qua AXI4-Full.
- Các pha của một tác vụ chạy tuần tự: nạp dữ liệu, tính, ghi kết quả.
- Không có: ngắt, C += A × B, ma trận lớn hơn 32 trong một tác vụ, dataflow Weight-Stationary, bộ nhớ ảo.

1.2. Quy ước

- M, N, K: A là M×K, B là K×N, C và Y là M×N. P: kích thước array vật lý (P×P PE). *word* = 32 bit; *beat* = một lần truyền AXI 32 bit; *tile* = khối P×P của C mà array tính trong một lượt.
- *Tác vụ*: từ khi lệnh START được chấp nhận đến khi STATUS báo DONE hoặc ERROR. *Bản chụp cấu hình*: bản sao các thanh ghi cấu hình, lấy đúng lúc START được chấp nhận và dùng suốt tác vụ.

1.3. Module

Bảng 1: Module trong `gemm_accel_top`

Module	Instance	Vai trò
`gemm_register_file`	`u_regfile`	AXI4-Lite slave; thanh ghi cấu hình, trạng thái, bộ đếm; chấp nhận START; giữ bản chụp cấu hình
`gemm_main_fsm`	`u_main_fsm`	Kiểm tra cấu hình; chạy lần lượt các pha của tác vụ; ra lệnh cho DMA và array; báo DONE/ERROR; điều phối soft reset
`gemm_dma_engine`	`u_dma`	AXI4-Full master; chép DRAM ↔ scratchpad theo lệnh LOAD_A, LOAD_B, LOAD_BIAS, STORE_C
`gemm_local_sram`	`u_sram`	Buffer A, B, C, BIAS, chia bank bên trong để phục vụ array và VPU mỗi chu kỳ
`gemm_systolic_array`	`u_array`	Mảng P×P PE (`gemm_pe`), làm lệch dữ liệu, bộ tuần tự tile, xả kết quả theo hàng
`gemm_vpu`	`u_vpu`	Hậu xử lý trên đường xả kết quả: bias, requantize, ReLU, kẹp INT8; đi thẳng khi tắt

Ngoài 6 module trên, `gemm_accel_top` có một bộ đồng bộ reset 2 flip-flop .

1.4. Tham số hệ thống

Bảng 2: Tham số hệ thống

Tham số	Giá trị	Ghi chú
`PE_ARRAY_SIZE` (P)	4 hoặc 8; mặc định 8	Kết quả không phụ thuộc P
`MAX_DIM`	32	Giới hạn M, N, K
`AXI_DATA_WIDTH`	32	Cả AXI4-Lite và AXI4-Full
`MAX_BURST_BEATS`	16 (mặc định)	Đặt theo giới hạn của bridge
`SRAM_RD_LAT`	1 chu kỳ	Cổng phía array và VPU
Buffer A / B / C / BIAS	256 / 256 / 1024 / 32 word	Tổng 6,125 KiB
`MMIO_BASE`	0x6000_0000, vùng 4 KiB	
`DRAM_BASE`	0x8000_0000	

Bảng 2: Tham số hệ thống (tiếp theo)

Tham số	Giá trị	Ghi chú
Burst outstanding	1 đọc + 1 ghi	

1.5. Tín hiệu giao tiếp

Tín hiệu giao tiếp của `gemm_accel_top` **đã khóa**: clock, reset, một cổng AXI4-Lite slave (17 tín hiệu) và một cổng AXI4-Full master (25 tín hiệu), tổng 44 port. Mọi thay đổi về sau, kể cả phần bổ sung ở mục 5, chỉ được thay đổi port nội bộ giữa các module, không được thêm, bớt hay đổi port ở hai bảng tín hiệu AXI ở trên.

1.5.1. Tham số

Bảng 3: Tham số của `gemm_accel_top`

Tham số	Mặc định	Ý nghĩa
`PE_ARRAY_SIZE`	8	P; chỉ nhận 4 hoặc 8
`MAX_BURST_BEATS`	16	Số beat tối đa mỗi burst, 1 … 256
`HAS_CHAIN`	0	1 nếu hiện thực chế độ chuỗi descriptor (mục 5.2)
`HAS_PINGPONG`	0	1 nếu hiện thực scratchpad ping-pong (mục 5.3); đòi hỏi `HAS_CHAIN` = 1

1.5.2. Danh sách tín hiệu

Bảng 4: Port clock, reset và AXI4-Lite slave (MMIO)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`clk`	1	Input	Clock của khối. Các thanh ghi cập nhật tại cạnh lên.
`rst_n`	1	Input	Reset ngoài tích cực mức thấp, kích hoạt bất đồng bộ. Top dùng hai flip-flop đồng bộ cạnh nhả trước khi phân phối reset tới các module.
`s_axi_awaddr`	32	Input	Địa chỉ byte của thanh ghi cần ghi. Chỉ giải mã bit [11:2]; bỏ qua bit [1:0]. Hợp lệ cùng AWVALID; giữ ổn định khi chờ bắt tay.
`s_axi_awvalid`	1	Input	Báo địa chỉ và thuộc tính yêu cầu ghi hợp lệ. Giữ bằng 1 và giữ nguyên payload cho đến khi AWREADY = 1.
`s_axi_awready`	1	Output	Báo bên nhận sẵn sàng nhận địa chỉ ghi. Bắt tay xảy ra tại cạnh lên khi AWVALID và AWREADY cùng bằng 1.
`s_axi_wdata`	32	Input	Word 32 bit cần ghi. Hợp lệ khi WVALID = 1; giữ nguyên trong lúc chờ WREADY.
`s_axi_wstrb`	4	Input	Cho phép ghi từng byte của thanh ghi RW. Bit b = 1 cho phép ghi byte b; bit b = 0 giữ byte đó. Các bit lệnh CTRL chỉ có hiệu lực khi bit 0 bằng 1.
`s_axi_wvalid`	1	Input	Báo dữ liệu ghi hợp lệ. Giữ valid và payload ổn định đến khi WREADY = 1.
`s_axi_wready`	1	Output	Báo bên nhận sẵn sàng nhận dữ liệu ghi. Giao dịch được nhận tại cạnh lên khi WVALID và WREADY cùng bằng 1.

Bảng 4: Port clock, reset và AXI4-Lite slave (MMIO) (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
s_axi_bresp	2	Output	Mã phản hồi ghi. Luôn bằng 2'b00 (OKAY), kể cả truy cập offset dự trữ. Hợp lệ cùng BVALID.
s_axi_bvalid	1	Output	Báo phản hồi ghi hợp lệ. Giữ valid và payload ổn định đến khi BREADY = 1.
s_axi_bready	1	Input	Báo bên nhận sẵn sàng nhận phản hồi ghi. Giao dịch được nhận tại cạnh lên khi BVALID và BREADY cùng bằng 1.
s_axi_araddr	32	Input	Địa chỉ byte của thanh ghi cần đọc. Chỉ giải mã bit [11:2]; bỏ qua bit [1:0]. Hợp lệ cùng ARVALID; giữ ổn định khi chờ bắt tay.
s_axi_arvalid	1	Input	Báo địa chỉ và thuộc tính yêu cầu đọc hợp lệ. Giữ bằng 1 và giữ nguyên payload cho đến khi ARREADY = 1.
s_axi_arready	1	Output	Báo bên nhận sẵn sàng nhận địa chỉ đọc. Bắt tay xảy ra tại cạnh lên khi ARVALID và ARREADY cùng bằng 1.
s_axi_rdata	32	Output	Word 32 bit đọc trả về. Hợp lệ khi RVALID = 1; giữ nguyên trong lúc chờ RREADY.
s_axi_rresp	2	Output	Mã phản hồi đọc. Luôn bằng 2'b00 (OKAY), kể cả truy cập offset dự trữ. Hợp lệ cùng RVALID.
s_axi_rvalid	1	Output	Báo dữ liệu đọc hợp lệ. Giữ valid và payload ổn định đến khi RREADY = 1.
s_axi_rready	1	Input	Báo bên nhận sẵn sàng nhận dữ liệu đọc. Giao dịch được nhận tại cạnh lên khi RVALID và RREADY cùng bằng 1.

Bảng 5: Port AXI4-Full master (DMA)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
m_axi_araddr	32	Output	Địa chỉ byte đầu tiên của burst đọc. Địa chỉ căn 4 byte. Hợp lệ cùng ARVALID; giữ ổn định khi chờ bắt tay.
m_axi_arlen	8	Output	Số beat của burst đọc trừ 1. Hợp lệ cùng ARVALID; số beat thực tế từ 1 đến MAX_BURST_BEATS và không vượt biên 4 KiB.
m_axi_arsize	3	Output	Kích thước mỗi beat của burst đọc. Luôn bằng 3'b010, tương ứng 4 byte.
m_axi_arburst	2	Output	Kiểu burst đọc. Luôn bằng 2'b01 (INCR); địa chỉ tăng 4 byte sau mỗi beat.
m_axi_arvalid	1	Output	Báo địa chỉ và thuộc tính yêu cầu đọc hợp lệ. Giữ bằng 1 và giữ nguyên payload cho đến khi ARREADY = 1.
m_axi_arready	1	Input	Báo bên nhận sẵn sàng nhận địa chỉ đọc. Bắt tay xảy ra tại cạnh lên khi ARVALID và ARREADY cùng bằng 1.
m_axi_rdata	32	Input	Word 32 bit đọc trả về. Hợp lệ khi RVALID = 1; giữ nguyên trong lúc chờ RREADY.

Bảng 5: Port AXI4-Full master (DMA) (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
m_axi_rresp	2	Input	Mã phản hồi beat đọc. Chỉ OKAY là thành công; mọi giá trị khác làm lệnh kết thúc có lỗi. Hợp lệ cùng RVALID.
m_axi_rlast	1	Input	Đánh dấu beat cuối burst đọc khi RVALID = 1. DMA so sánh vị trí với ARLEN + 1; đến sớm hoặc muộn đều là lỗi.
m_axi_rvalid	1	Input	Báo dữ liệu đọc hợp lệ. Giữ valid và payload ổn định đến khi RREADY = 1.
m_axi_rready	1	Output	Báo bên nhận sẵn sàng nhận dữ liệu đọc. Giao dịch được nhận tại cạnh lên khi RVALID và RREADY cùng bằng 1.
m_axi_awaddr	32	Output	Địa chỉ byte đầu tiên của burst ghi. Địa chỉ căn 4 byte. Hợp lệ cùng AWVALID; giữ ổn định khi chờ bắt tay.
m_axi_awlen	8	Output	Số beat của burst ghi trừ 1. Hợp lệ cùng AWVALID; số beat thực tế từ 1 đến MAX_BURST_BEATS và không vượt biên 4 KiB.
m_axi_awsize	3	Output	Kích thước mỗi beat của burst ghi. Luôn bằng 3'b010, tương ứng 4 byte.
m_axi_awburst	2	Output	Kiểu burst ghi. Luôn bằng 2'b01 (INCR); địa chỉ tăng 4 byte sau mỗi beat.
m_axi_awvalid	1	Output	Báo địa chỉ và thuộc tính yêu cầu ghi hợp lệ. Giữ bằng 1 và giữ nguyên payload cho đến khi AWREADY = 1.
m_axi_awready	1	Input	Báo bên nhận sẵn sàng nhận địa chỉ ghi. Bắt tay xảy ra tại cạnh lên khi AWVALID và AWREADY cùng bằng 1.
m_axi_wdata	32	Output	Word 32 bit cần ghi. Hợp lệ khi WVALID = 1; giữ nguyên trong lúc chờ WREADY.
m_axi_wstrb	4	Output	Mặt nạ byte ghi. Luôn bằng 4'b1111 vì DMA ghi nguyên word 32 bit.
m_axi_wlast	1	Output	Đánh dấu beat cuối burst ghi khi WVALID = 1. Bằng 1 ở đúng beat thứ AWLEN + 1 và giữ nguyên trong lúc chờ WREADY.
m_axi_wvalid	1	Output	Báo dữ liệu ghi hợp lệ. Giữ valid và payload ổn định đến khi WREADY = 1.
m_axi_wready	1	Input	Báo bên nhận sẵn sàng nhận dữ liệu ghi. Giao dịch được nhận tại cạnh lên khi WVALID và WREADY cùng bằng 1.
m_axi_bresp	2	Input	Mã phản hồi burst ghi. Chỉ OKAY là thành công; mọi giá trị khác làm lệnh kết thúc có lỗi. Hợp lệ cùng BVALID.
m_axi_bvalid	1	Input	Báo phản hồi ghi hợp lệ. Giữ valid và payload ổn định đến khi BREADY = 1.
m_axi_bready	1	Output	Báo bên nhận sẵn sàng nhận phản hồi ghi. Giao dịch được nhận tại cạnh lên khi BVALID và BREADY cùng bằng 1. DMA luôn giữ BREADY = 1.

Port top-level

- Tín hiệu giao tiếp của `gemm_accel_top` đúng tên, độ rộng và chiều như hai bảng tín hiệu AXI ở trên; không có port nào khác, kể cả ngắt
- Danh sách port này cố định cho mọi giá trị tham số đã liệt kê ở trên
- Giá trị tham số ngoài miền cho phép phải bị chặn lúc elaboration
- Kênh đọc AXI (AR, R) và kênh ghi (AW, W, B) do DMA điều khiển; không có đường tổ hợp từ input tới output trên cùng một cổng AXI
- `gemm_accel_top` là Chisel BlackBox, nối hệ thống qua cầu TLToAXI4 (MMIO) và AXI4ToTL (DMA). Tín hiệu AXI phụ (ID, LOCK, CACHE, PROT, QOS) do `GemmWrapper.scala` cấp

2. Đặc tả thanh ghi

Firmware ghi A, B (và bias) vào DRAM, ghi kích thước, địa chỉ và tham số hậu xử lý vào thanh ghi, ghi START, rồi đọc STATUS cho tới khi DONE hoặc ERROR bằng 1. Register map dưới đây đã khóa: phần bổ sung ở mục 5 dùng các offset và bit dành cho chế độ chuỗi.

2.1. Tổng hợp thanh ghi

Địa chỉ tuyệt đối = MMIO_BASE + offset; vùng MMIO 4 KiB tại 0x6000_0000. Thanh ghi căn word; phần cứng chỉ giải mã bit địa chỉ [11:2]. RW: đọc/ghi theo byte; RO: chỉ đọc, ghi bị bỏ qua; W: ghi lệnh, đọc luôn bằng 0. Không có bit RW1C. Các offset không liệt kê đọc 0, ghi bị bỏ qua, phản hồi OKAY.

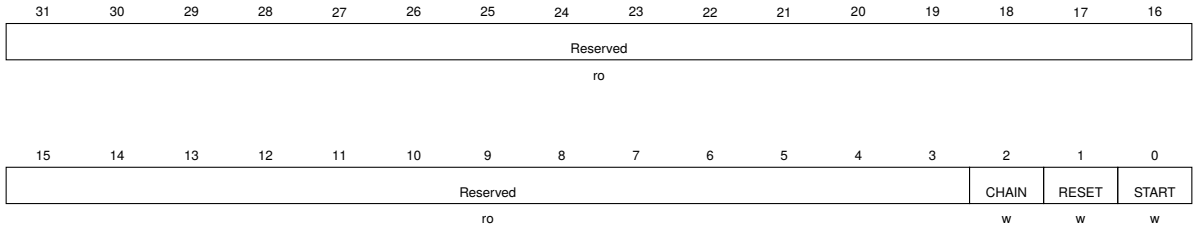
Bảng 6: Tổng hợp thanh ghi

Offset	Tên viết tắt	Tên thanh ghi
0x00	CTRL_REG	Thanh ghi điều khiển
0x04	STATUS_REG	Thanh ghi trạng thái
0x08	MATRIX_DIM	Kích thước ma trận
0x0C	BASE_ADDR_IN	Địa chỉ byte của A
0x10	BASE_ADDR_WEIGHT	Địa chỉ byte của B
0x14	BASE_ADDR_OUT	Địa chỉ byte vùng kết quả C hoặc Y
0x18	POST_CTRL	Điều khiển hậu xử lý
0x1C	BIAS_ADDR	Địa chỉ byte vector bias; bỏ qua khi POST_EN = 0
0x20	QUANT_M0	Hệ số lượng tử
0x24	QUANT_SHIFT	Lượng dịch lượng tử
0x28	HW_CONFIG	Cấu hình phần cứng
0x2C	PERF_TOTAL	Tổng số chu kỳ BUSY của tác vụ
0x30	PERF_LOAD	Số chu kỳ LOAD, không tính FETCH
0x34	PERF_COMPUTE	Số chu kỳ COMPUTE
0x38	PERF_STORE	Số chu kỳ STORE_C
0x40	DESC_PTR	Con trỏ descriptor đầu
0x44	ERR_DESC_ADDR	Địa chỉ descriptor lỗi
0x48	DESC_DONE_CNT	Số descriptor hoàn tất
Các địa chỉ khác	—	Reserved

2.2. Thanh ghi điều khiển (CTRL_REG)

Địa chỉ offset: 0x00

Giá trị reset: 0x0000_0000



Bảng 7: Thanh ghi điều khiển

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
31:3	Reserved	RO	29'h0	Đọc 0; ghi bị bỏ qua.
2	START_CHAIN	W	1'h0	Ghi 1 yêu cầu chạy chuỗi descriptor. Khi không hỗ trợ chuỗi, tác vụ kết thúc lỗi mã 5.
1	SOFT_RESET	W	1'h0	Ghi 1 yêu cầu soft reset; ưu tiên cao hơn cả hai lệnh START.
0	START	W	1'h0	Ghi 1 yêu cầu chạy một GEMM. Chỉ nhận khi rảnh; khi bận bỏ qua. Ghi đồng thời START và START_CHAIN khi không có SOFT_RESET tạo lỗi mã 1.

2.3. Thanh ghi trạng thái (STATUS_REG)

Địa chỉ offset: 0x04

Giá trị reset: 0x0000_0000



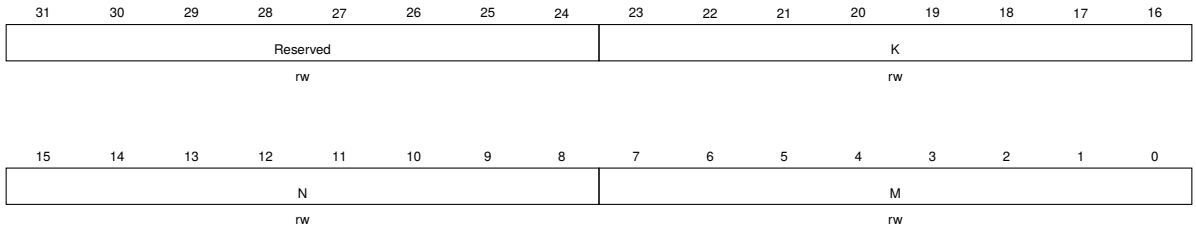
Bảng 8: Thanh ghi trạng thái

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
31:8	Reserved	RO	24'h0	Đọc 0; ghi bị bỏ qua.
7:4	ERR_CODE	RO	4'h0	Mã lỗi của tác vụ; bằng 0 khi ERROR = 0. Bảng mã lỗi ở dưới.
3	MODE	RO	1'h0	0: tác vụ đơn; 1: chuỗi descriptor. Giữ chế độ tác vụ gần nhất.
2	ERROR	RO	1'h0	Cờ lỗi sticky; xóa khi nhận tác vụ mới hoặc reset.
1	BUSY	RO	1'h0	Tác vụ hoặc chuỗi soft reset đang chạy. Về 0 cùng cạnh đặt DONE hoặc ERROR.
0	DONE	RO	1'h0	Sticky; chỉ đặt khi toàn bộ kết quả đã ghi DRAM thành công. Xóa khi nhận tác vụ mới hoặc reset.

2.4. Kích thước ma trận (MATRIX_DIM)

Địa chỉ offset: 0x08

Giá trị reset: 0x0000_0000



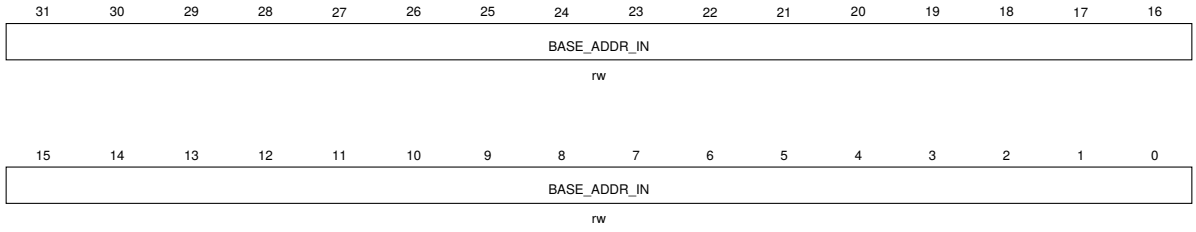
Bảng 9: Kích thước ma trận

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
31:24	Reserved	RW	8'h0	Lưu và đọc lại; phải bằng 0 khi khởi chạy.
23:16	K	RW	8'h0	Số cột A/số hàng B; 4, 8, …, 32.
15:8	N	RW	8'h0	Số cột B/C; 4, 8, …, 32.
7:0	M	RW	8'h0	Số hàng A/C; 1 … 32.

2.5. Địa chỉ byte của A (BASE_ADDR_IN)

Địa chỉ offset: 0x0C

Giá trị reset: 0x0000_0000



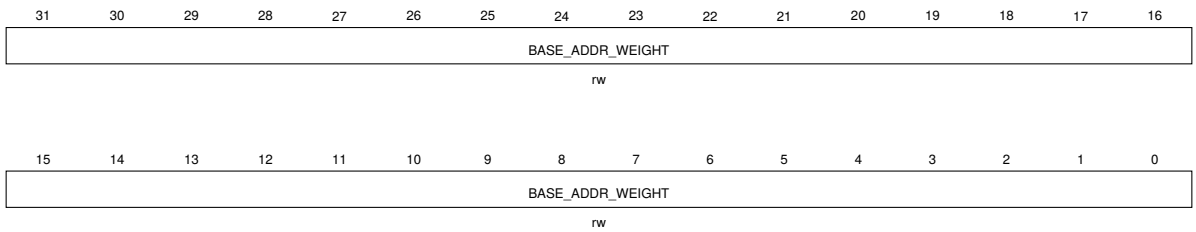
Bảng 10: Địa chỉ byte của A

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
31:0	BASE_ADDR_IN	RW	32'h0	Địa chỉ byte của A; căn 4 byte khi được sử dụng.

2.6. Địa chỉ byte của B (BASE_ADDR_WEIGHT)

Địa chỉ offset: 0x10

Giá trị reset: 0x0000_0000



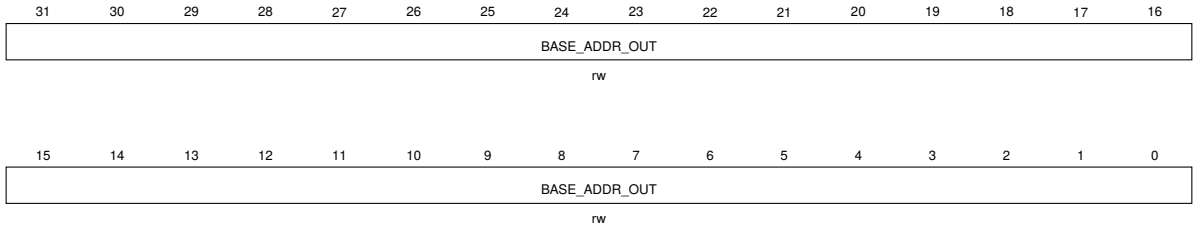
Bảng 11: Địa chỉ byte của B

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
31:0	BASE_ADDR_WEIGHT	RW	32'h0	Địa chỉ byte của B; căn 4 byte khi được sử dụng.

2.7. Địa chỉ byte vùng kết quả C hoặc Y (BASE_ADDR_OUT)

Địa chỉ offset: 0x14

Giá trị reset: 0x0000_0000



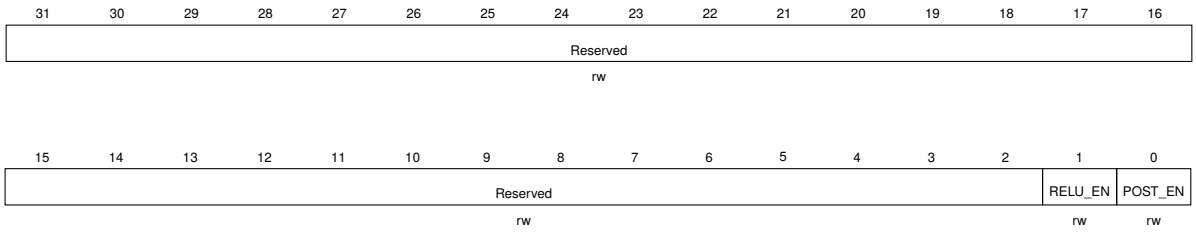
Bảng 12: Địa chỉ byte vùng kết quả C hoặc Y

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
31:0	BASE_ADDR_OUT	RW	32'h0	Địa chỉ byte vùng kết quả C hoặc Y; căn 4 byte khi được sử dụng.

2.8. Điều khiển hậu xử lý (POST_CTRL)

Địa chỉ offset: 0x18

Giá trị reset: 0x0000_0000



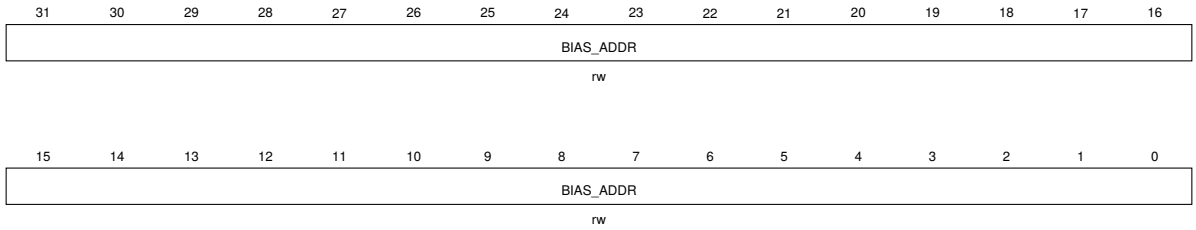
Bảng 13: Điều khiển hậu xử lý

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
31:2	Reserved	RW	30'h0	Lưu và đọc lại; phải bằng 0 khi khởi chạy.
1	RELU_EN	RW	1'h0	Bật ReLU; chỉ hợp lệ khi POST_EN = 1.
0	POST_EN	RW	1'h0	0: kết quả INT32. 1: cộng bias, requantize, tùy chọn ReLU và kẹp về INT8.

2.9. Địa chỉ byte vector bias; bỏ qua khi POST_EN = 0 (BIAS_ADDR)

Địa chỉ offset: 0x1C

Giá trị reset: 0x0000_0000



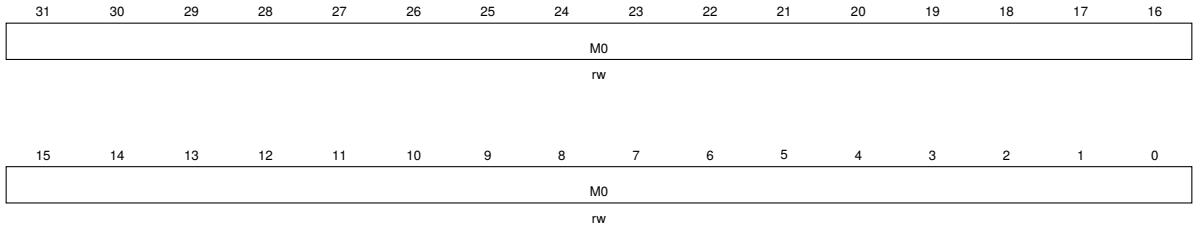
Bảng 14: Địa chỉ byte vector bias; bỏ qua khi POST_EN = 0

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
31:0	BIAS_ADDR	RW	32'h0	Địa chỉ byte vector bias; bỏ qua khi POST_EN = 0; căn 4 byte khi được sử dụng.

2.10. Hệ số lượng tử (QUANT_M0)

Địa chỉ offset: 0x20

Giá trị reset: 0x0000_0000



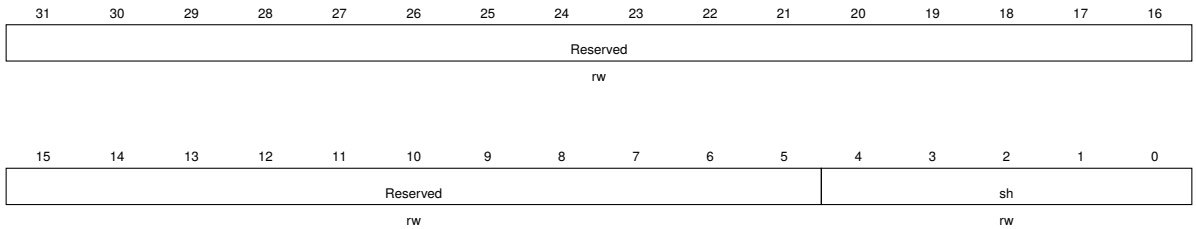
Bảng 15: Hệ số lượng tử

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
31:0	M0	RW	32'h0	Số nguyên có dấu INT32, mọi giá trị hợp lệ; bỏ qua khi POST_EN = 0.

2.11. Lượng dịch lượng tử (QUANT_SHIFT)

Địa chỉ offset: 0x24

Giá trị reset: 0x0000_0000



Bảng 16: Lượng dịch lượng tử

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
31:5	Reserved	RW	27'h0	Lưu và đọc lại; phải bằng 0 khi POST_EN = 1.
4:0	sh	RW	5'h0	Số không dấu 0 … 31; tổng lượng dịch là 31 + sh.

2.12. Cấu hình phần cứng (HW_CONFIG)

Địa chỉ offset: 0x28

Giá trị reset: {8'h10, 6'b0, HAS_PINGPONG, HAS_CHAIN, 8'd32, P[7:0]}



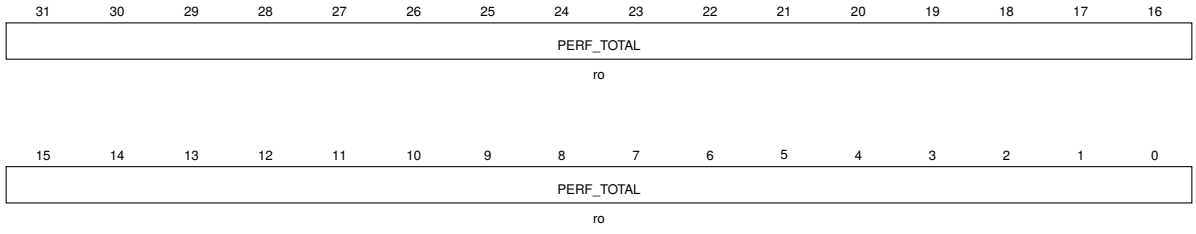
Bảng 17: Cấu hình phần cứng

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
31:24	SPEC_VER	RO	0x10	Phiên bản giao diện phần cứng v1.0; việc biên tập tài liệu không thay đổi giá trị này.
23:18	Reserved	RO	6'h0	Đọc 0; ghi bị bỏ qua.
17	HAS_PINGPONG	RO	tham số	Có hai bank mỗi buffer.
16	HAS_CHAIN	RO	tham số	Có chế độ chuỗi descriptor.
15:8	MAX_DIM	RO	32	Kích thước ma trận tối đa.
7:0	P	RO	tham số	Kích thước array; 4 hoặc 8.

2.13. Tổng số chu kỳ BUSY của tác vụ (PERF_TOTAL)

Địa chỉ offset: 0x2C

Giá trị reset: 0x0000_0000



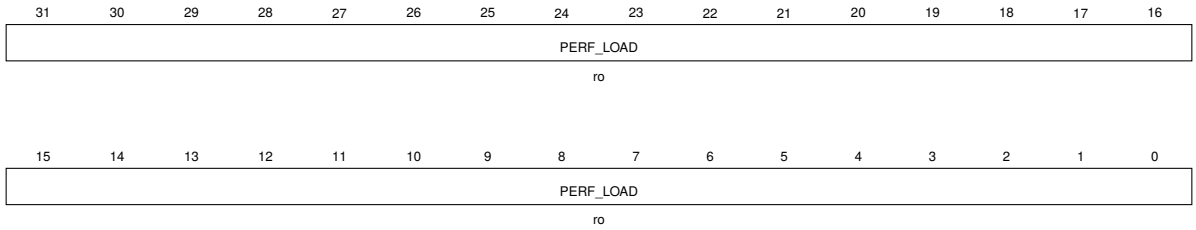
Bảng 18: Tổng số chu kỳ BUSY của tác vụ

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
31:0	PERF_TOTAL	RO	32'h0	Tổng số chu kỳ BUSY của tác vụ. Xóa khi nhận tác vụ hoặc reset; bão hòa tại 0xFFFF_FFFF, giữ sau khi kết thúc.

2.14. Số chu kỳ LOAD, không tính FETCH (PERF_LOAD)

Địa chỉ offset: 0x30

Giá trị reset: 0x0000_0000



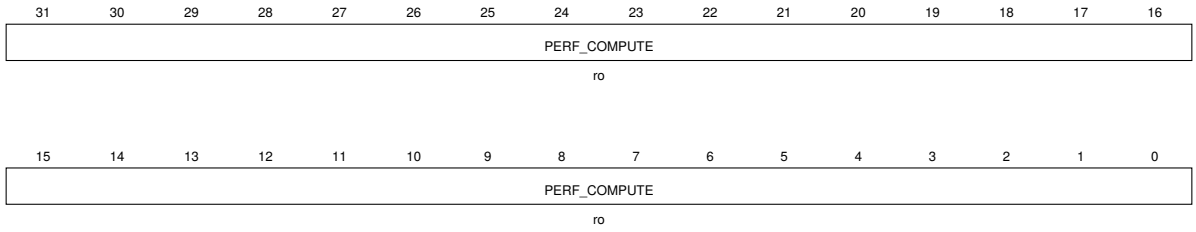
Bảng 19: Số chu kỳ LOAD, không tính FETCH

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
31:0	PERF_LOAD	RO	32'h0	Số chu kỳ LOAD, không tính FETCH. Xóa khi nhận tác vụ hoặc reset; bão hòa tại 0xFFFF_FFFF, giữ sau khi kết thúc.

2.15. Số chu kỳ COMPUTE (PERF_COMPUTE)

Địa chỉ offset: 0x34

Giá trị reset: 0x0000_0000



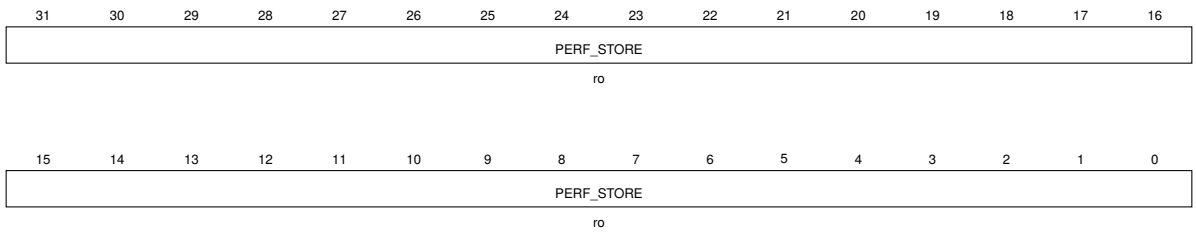
Bảng 20: Số chu kỳ COMPUTE

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
31:0	PERF_COMPUTE	RO	32'h0	Số chu kỳ COMPUTE. Xóa khi nhận tác vụ hoặc reset; bão hòa tại 0xFFFF_FFFF, giữ sau khi kết thúc.

2.16. Số chu kỳ STORE_C (PERF_STORE)

Địa chỉ offset: 0x38

Giá trị reset: 0x0000_0000



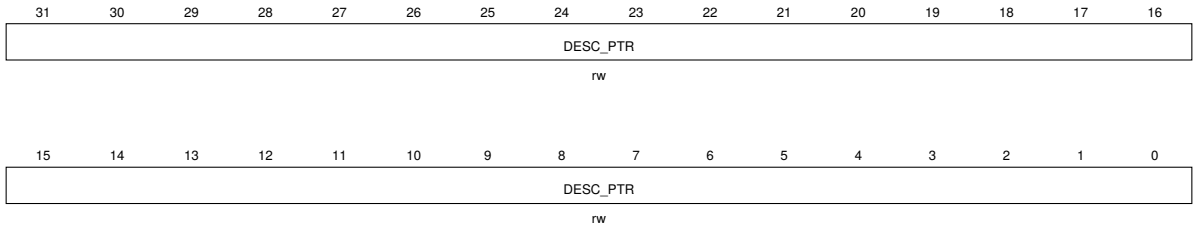
Bảng 21: Số chu kỳ STORE_C

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
31:0	PERF_STORE	RO	32'h0	Số chu kỳ STORE_C. Xóa khi nhận tác vụ hoặc reset; bão hòa tại 0xFFFF_FFFF, giữ sau khi kết thúc.

2.17. Con trỏ descriptor đầu (DESC_PTR)

Địa chỉ offset: 0x40

Giá trị reset: 0x0000_0000



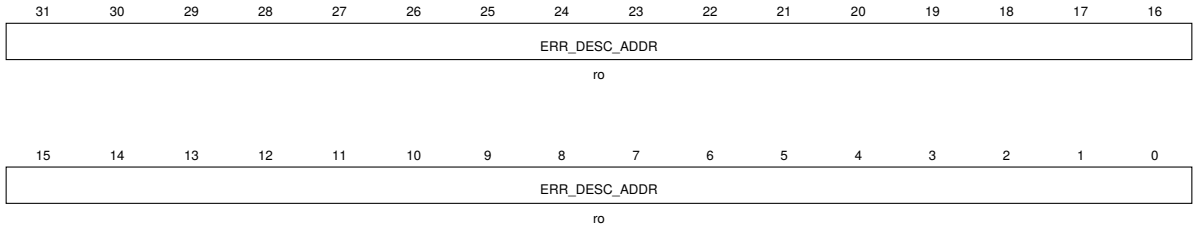
Bảng 22: Con trỏ descriptor đầu

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
31:0	DESC_PTR	RW	32'h0	Địa chỉ byte descriptor đầu; căn 64 byte. Khi HAS_CHAIN = 0: đọc 0, ghi bị bỏ qua.

2.18. Địa chỉ descriptor lỗi (ERR_DESC_ADDR)

Địa chỉ offset: 0x44

Giá trị reset: 0x0000_0000



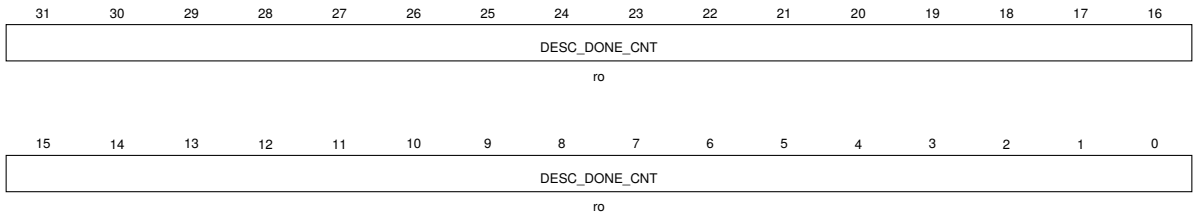
Bảng 23: Địa chỉ descriptor lỗi

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
31:0	ERR_DESC_ADDR	RO	32'h0	Địa chỉ descriptor có lỗi sớm nhất theo thứ tự chuỗi. Xóa khi nhận tác vụ/reset; bằng 0 ở chế độ đơn hoặc khi không có chuỗi.

2.19. Số descriptor hoàn tất (DESC_DONE_CNT)

Địa chỉ offset: 0x48

Giá trị reset: 0x0000_0000



Bảng 24: Số descriptor hoàn tất

Bit	Tên	Kiểu	Giá trị mặc định	Mô tả
31:0	DESC_DONE_CNT	RO	32'h0	Tăng khi STORE_C của descriptor thành công. Xóa khi nhận tác vụ/reset; bằng 0 ở chế độ đơn hoặc khi không có chuỗi.

2.20. Mã lỗi

Bảng 25: Mã lỗi `ERR_CODE` (STATUS[7:4])

Giá trị	Ý nghĩa	Khi nào xuất hiện
0	Không lỗi	ERROR = 0
1	Cấu hình sai	Bản chụp vi phạm điều kiện hợp lệ của thanh ghi; hoặc START và START_CHAIN cùng được ghi 1
2	Lỗi AXI khi đọc	LOAD_A, LOAD_B hoặc LOAD_BIAS gặp `rresp` $\neq$ OKAY hoặc `rlast` sai vị trí
3	Lỗi AXI khi ghi	STORE_C gặp `bresp` $\neq$ OKAY
4	Lỗi AXI khi đọc descriptor	Chỉ có với phần bổ sung (mục 5.2)
5	Tính năng không có	START_CHAIN khi phần cứng không có chế độ chuỗi
6–15	Dự trữ	Không bao giờ xuất hiện

2.21. Yêu cầu thanh ghi

Thanh ghi

- Offset, tên, quyền truy cập và giá trị reset theo register map ở trên
- CTRL[0] START: ghi 1 là yêu cầu bắt đầu tác vụ; bit tự xóa, đọc luôn ra 0; ghi 0 không có tác dụng
- CTRL[1] SOFT_RESET: ghi 1 là yêu cầu soft reset (mục 4.5); tự xóa
- CTRL[2] START_CHAIN: khi `HAS_CHAIN` = 0, lệnh vẫn được chấp nhận như một tác vụ và kết thúc ngay với ERROR, `ERR_CODE` = 5, không truy cập bus
- Một lần ghi CTRL có SOFT_RESET = 1: chỉ SOFT_RESET có hiệu lực. Không có SOFT_RESET mà START = START_CHAIN = 1: tác vụ được chấp nhận và kết thúc ngay với `ERR_CODE` = 1
- STATUS.MODE = 0 với tác vụ START, = 1 với tác vụ START_CHAIN; giữ giá trị của tác vụ gần nhất. `ERR_CODE` = 0 khi ERROR = 0
- Thanh ghi R/W được ghi bất kỳ lúc nào, kể cả khi BUSY = 1. Đọc lại trả giá trị CPU đã ghi, không phải bản chụp. Giá trị mới áp dụng từ lần START được chấp nhận tiếp theo
- Thanh ghi R/W lưu đủ 32 bit, kể cả bit dự trữ, để phát hiện cấu hình có trường dự trữ khác 0. Bit dự trữ của CTRL và STATUS: ghi bị bỏ qua, đọc ra 0. Offset dự trữ: đọc ra 0, ghi bị bỏ qua. Ghi vào thanh ghi chỉ đọc bị bỏ qua

- `rst_n` đưa mọi thanh ghi về giá trị reset. Soft reset giữ nguyên các thanh ghi R/W; xóa STATUS và `PERF_*`
- `PERF_*` về 0 ở chu kỳ START được chấp nhận, đếm trong lúc BUSY = 1, dừng khi tác vụ kết thúc, bão hòa ở 0xFFFF_FFFF
- Bản chụp cấu hình gồm `MATRIX_DIM`, ba `BASE_ADDR_*`, `POST_CTRL`, `BIAS_ADDR`, `QUANT_M0`, `QUANT_SHIFT`; lấy ở chu kỳ START được chấp nhận và giữ ổn định tới hết tác vụ
- Cấu hình hợp lệ khi và chỉ khi: M, N, K trong miền $1 \leq M \leq 32$ và N, K thuộc {4, 8, …, 32}; `MATRIX_DIM[31:24]` = 0; `BASE_ADDR_IN`, `BASE_ADDR_WEIGHT`, `BASE_ADDR_OUT` có bit [1:0] = 0; `POST_CTRL[31:2]` = 0; `RELU_EN` = 1 kéo theo `POST_EN` = 1; nếu `POST_EN` = 1 thì `BIAS_ADDR[1:0]` = 0 và `QUANT_SHIFT[31:5]` = 0. Khi `POST_EN` = 0, `BIAS_ADDR`, `QUANT_M0`, `QUANT_SHIFT` bị bỏ qua

2.22. Ngữ nghĩa tác vụ

Tác vụ

- START chỉ được chấp nhận nếu BUSY = 0 tại chu kỳ Register File thực hiện lệnh ghi CTRL. START khi BUSY = 1 bị bỏ qua, không đổi trạng thái, không báo lỗi
- Ngay chu kỳ chấp nhận: chụp cấu hình ; xóa DONE, ERROR, `ERR_CODE`, `PERF_*`; đặt MODE. BUSY = 1 từ chu kỳ kế tiếp
- Register File phát `start_pulse` và chụp cấu hình trước khi trả phản hồi B cho lệnh ghi CTRL; mọi lần đọc STATUS sau khi lệnh ghi hoàn tất đều thấy tác vụ mới
- DONE = 1 chỉ khi STORE_C đã nhận phản hồi B cuối cùng là OKAY, tức toàn bộ kết quả đã nằm trong DRAM
- ERROR = 1 khi gặp một lỗi đã liệt kê ở trên; sau lỗi không pha mới nào được bắt đầu; nội dung vùng đích trong DRAM không xác định
- BUSY về 0 ở đúng cạnh clock mà DONE hoặc ERROR được đặt. Sau khi chấp nhận và cho tới tác vụ kế tiếp hoặc soft reset, mọi lần đọc STATUS thấy đúng một trong ba bit BUSY, DONE, ERROR bằng 1
- DONE và ERROR giữ giá trị (sticky); chỉ bị xóa khi tác vụ mới được chấp nhận, khi soft reset hoặc khi `rst_n`
- Sau DONE hoặc ERROR, firmware được phép ghi START ngay, không cần soft reset
- Tín hiệu "xong" nội bộ (xong một lệnh DMA, `pe_done`) không hiện ra cho CPU; chỉ STATUS.DONE báo toàn bộ tác vụ xong

2.23. Quy tắc cho firmware

Firmware

- Trước lệnh ghi START, firmware thực hiện `fencerw,rw`, để mọi lệnh ghi A, B, bias vào DRAM đến trước lệnh MMIO
- Firmware chờ bằng polling cho tới khi DONE hoặc ERROR bằng 1, rồi thực hiện `fencerw,rw` trước khi đọc kết quả
- Đường DMA được giả định nhất quán cache với CPU. Nếu hệ thống tích hợp không bảo đảm điều đó, firmware phải flush dòng cache của A, B, bias trước START và hủy dòng cache của vùng đích sau DONE
- Firmware không ghi vào vùng DRAM của tác vụ đang chạy

```
#define GEMM_BASE  0x60000000u
#define REG(off)   (*(volatile uint32_t *)(GEMM_BASE + (off)))
#define ST_DONE    (1u << 0)
#define ST_ERROR   (1u << 2)

int gemm_run(uint32_t a, uint32_t b, uint32_t c, uint32_t M, uint32_t N,
             uint32_t K, int post, int relu, uint32_t bias,
             int32_t m0, uint32_t sh) {
    REG(0x08) = (K << 16) | (N << 8) | M;     /* MATRIX_DIM   */
    REG(0x0C) = a;  REG(0x10) = b;  REG(0x14) = c;
    REG(0x18) = (post ? 1u : 0u) | (relu ? 2u : 0u);
    REG(0x1C) = bias;  REG(0x20) = (uint32_t)m0;  REG(0x24) = sh;
    asm volatile ("fence rw,rw");             /* (Trước lệnh ghi START, firmware thực hiện \sig{fence rw,rw},
    REG(0x00) = 1u;                           /* START */
    uint32_t s;
    do { s = REG(0x04); } while (!(s & (ST_DONE | ST_ERROR)));
    asm volatile ("fence rw,rw");             /* (Firmware chờ bằng polling cho tới khi DONE hoặc ERROR bằng
    return (s & ST_ERROR) ? (int)((s >> 4) & 0xFu) : 0;
}
```

3. Mô tả chức năng và dữ liệu

Mỗi tác vụ tính C = A $\times$ B trên INT8, tích lũy INT32. Khi POST_EN = 0, C (INT32) được ghi ra DRAM. Khi POST_EN = 1, VPU biến C thành Y = kẹp(ReLU(requantize(C + bias))) ở INT8 và Y được ghi ra DRAM. Mọi phép tính là số nguyên và kết quả phải khớp từng bit với mô hình tham chiếu ở mục 3.4.

3.1. Phép toán

Phép toán

- $C[i][j] = \sum_{k=0}^{K-1} A[i][k] \cdot B[k][j]$, với 0 $\leq$ i < M, 0 $\leq$ j < N
- A, B là INT8 bù 2. C và bias là INT32 bù 2. Y là INT8 bù 2. Tích INT8 $\times$ INT8 tính đủ 16 bit có dấu; cộng dồn trên INT32, wrap mod 2^{32} nếu tràn
- Khi POST_EN = 1, với mỗi phần tử:

  $acc = C[i][j] + bias[j]$ (INT32, wrap mod 2^{32})
  $prod = acc \cdot M_0$ (tích có dấu 64 bit)
  $y = (prod + 2^{30+sh}) \gg (31 + sh)$ (dịch số học, 64 bit)
  nếu RELU_EN = 1: $y = \max(y, 0)$
  $Y[i][j] = \min(\max(y, -128), 127)$

- M_0 (QUANT_M0) là INT32 có dấu, mọi giá trị đều hợp lệ. sh (QUANT_SHIFT) là số không dấu 0 … 31. Một cặp (M_0, sh) dùng cho cả tác vụ. bias là vector N phần tử INT32, cộng theo cột
- Kết quả ghi đè vùng đích trong DRAM; accelerator không đọc giá trị cũ của vùng đích
- Chỉ hỗ trợ nhân ma trận và hậu xử lý đã mô tả; không có C += A $\times$ B, softmax hoặc pooling.
- Kết quả không phụ thuộc P: P = 4 và P = 8 cho nội dung DRAM giống hệt từng bit

> **HỆ QUẢ**
> $|a \cdot b| \leq 2^{14}$ và K $\leq$ 32 nên $|C| \leq 2^{19}$: C không bao giờ tràn. $|prod| \leq 2^{62}$ và $2^{30+sh} \leq 2^{61}$ nên phép tính hậu xử lý 64 bit không bao giờ tràn. Làm tròn là nửa lên (về phía +∞). Phần mềm nên chọn M_0 trong $[2^{30}, 2^{31})$ để giữ độ chính xác.

3.2. Kích thước ma trận

Miền kích thước: $1 \leq M \leq 32$; N, K thuộc 4, 8, …, 32.

Ma trận có N hoặc K không chia hết cho 4 phải được phần mềm đệm 0 lên bội của 4; cột thừa của kết quả bị phần mềm bỏ qua. Toàn bộ A, B, bias và C của một tác vụ nằm trong scratchpad suốt tác vụ.

3.3. Bố trí dữ liệu trong DRAM

Bảng 26: Địa chỉ byte của phần tử

Ma trận	Địa chỉ	Kích thước vùng
A	`BASE_ADDR_IN` + i·K + k	M·K byte
B	`BASE_ADDR_WEIGHT` + k·N + j	K·N byte
bias	`BIAS_ADDR` + 4j	4·N byte
C (INT32)	`BASE_ADDR_OUT` + 4·(i·N + j)	4·M·N byte
Y (INT8)	`BASE_ADDR_OUT` + i·N + j	M·N byte

Bố trí dữ liệu

- Chế độ đơn dùng row-major, các hàng liền nhau; địa chỉ phần tử theo bảng Địa chỉ byte của phần tử
- Little-endian: byte tại địa chỉ a nằm ở bit [8·(a mod 4)+7 : 8·(a mod 4)] của word 32 bit; INT32 là little-endian
- Mọi địa chỉ cơ sở căn 4 byte. Vì N, K là bội của 4, mọi hàng bắt đầu và kết thúc đúng biên word, nên `m_axi_wstrb` luôn bằng 4'b1111
- Vùng đích không được chồng lên vùng A, B hoặc bias; A, B, bias được phép chồng nhau. Vi phạm → kết quả không xác định. Phần cứng không kiểm tra
- Phần cứng không kiểm tra vùng nhớ có nằm trong DRAM không; bus trả lỗi thì tác vụ kết thúc với ERROR (mục 4.4)

3.4. Mô hình tham chiếu

```
import numpy as np

def gemm(A, B):                        # A: int8 MxK, B: int8 KxN
    return (A.astype(np.int64) @ B.astype(np.int64)).astype(np.int32)

def post(C, bias, M0, sh, relu):       # C: int32 MxN, bias: int32 N
    acc  = (C.astype(np.int64) + bias).astype(np.int32).astype(np.int64)
    prod = acc * np.int64(M0)
    y    = (prod + (np.int64(1) << (30 + sh))) >> (31 + sh)
    if relu:
        y = np.maximum(y, 0)
    return np.clip(y, -128, 127).astype(np.int8)
```

4. Luồng vận hành

Một tác vụ chạy các pha theo thứ tự CHECK_CFG → LOAD_BIAS (nếu bật hậu xử lý) → LOAD_A → LOAD_B → COMPUTE → STORE_C. Mỗi lúc chỉ một pha chạy, và mỗi pha có đúng một khối làm chính. Pha sau chỉ bắt đầu khi pha trước thành công.

4.1. Các pha của một tác vụ

Bảng 27: Các pha

Pha	Khối chính	Làm gì	Xong khi
CHECK_CFG	Main FSM	Kiểm tra kích thước, căn địa chỉ và các trường dự trữ theo điều kiện hợp lệ của thanh ghi. Sai: lỗi mã 1; không truy cập bus hoặc scratchpad.	Kiểm tra xong
LOAD_BIAS	DMA	Chép N word bias từ DRAM vào buffer BIAS. Chỉ có khi `POST_EN` = 1	SRAM nhận word cuối
LOAD_A	DMA	Chép A (M hàng × K/4 word) vào buffer A	SRAM nhận word cuối
LOAD_B	DMA	Chép B (K hàng × N/4 word) vào buffer B	SRAM nhận word cuối
COMPUTE	Array, VPU	Array tính mọi tile; mỗi hàng kết quả đi qua VPU (hậu xử lý hoặc đi thẳng) rồi ghi vào buffer C	`pe_done`
STORE_C	DMA	Chép C (M·N word) hoặc Y (M·N/4 word) từ buffer C ra DRAM	Phản hồi B cuối là OKAY

4.2. Luồng một tác vụ

Có ba mức "xong": `o_done` của DMA báo xong một lệnh DMA; `pe_done` báo toàn bộ kết quả đã nằm trong buffer C; STATUS.DONE báo toàn bộ kết quả đã nằm trong DRAM. Chỉ mức cuối cùng hiện ra cho CPU.

4.3. Quyền truy cập scratchpad theo pha

Bảng 28: Ai truy cập buffer trong mỗi pha

Pha	A	B	BIAS	C
IDLE, CHECK_CFG	—	—	—	—
LOAD_BIAS	—	—	DMA ghi	—
LOAD_A	DMA ghi	—	—	—
LOAD_B	—	DMA ghi	—	—
COMPUTE	Array đọc	Array đọc	VPU đọc	VPU ghi
STORE_C	—	—	—	DMA đọc

Quyền truy cập

- Mỗi buffer chỉ có tối đa một bên truy cập trong mỗi pha, theo các quy tắc địa chỉ và giao tiếp bên dưới. Main FSM bảo đảm điều này bằng cách chạy tuần tự các pha. Vì vậy mỗi bank vật lý chỉ cần SRAM một cổng
- Module SRAM có assertion (mô phỏng) phát hiện truy cập sai bên trong một pha

4.4. Xử lý lỗi

Lỗi

- Cấu hình sai: CHECK_CFG kết thúc tác vụ với `ERR_CODE` = 1; không có giao dịch AXI nào, không truy cập scratchpad
- Lỗi AXI trong một lệnh DMA: DMA ngừng phát burst mới, nhận đủ beat R hoặc phản hồi B của burst đã phát, rồi báo `o_error`. Main FSM kết thúc tác vụ với `ERR_CODE` = 2 (LOAD_*) hoặc 3 (STORE_C) và không phát lệnh nào nữa
- Array và VPU không có đầu ra lỗi

4.5. Reset và soft reset

Reset

- `rst_n` tích cực mức thấp, bất đồng bộ. `gemm_accel_top` có bộ đồng bộ 2 flip-flop (bật bất đồng bộ, nhả đồng bộ với `clk`); đầu ra `rst_n_sync` cấp cho mọi module
- `rst_n_sync` = 0 đưa mọi trạng thái về giá trị reset. Nội dung scratchpad không bị xóa, coi là không xác định
- Soft reset không đi vào chân reset; nó là xung xóa đồng bộ `soft_clr` một chu kỳ do Main FSM phát tới Register File, DMA, SRAM, Array, VPU
- Phạm vi soft reset: FSM, DMA, Array, VPU, phần điều khiển SRAM; STATUS và `PERF_*`. Thanh ghi R/W giữ nguyên. Nội dung scratchpad không xác định
- Nếu DMA đang chạy lệnh: FSM bật `i_dma_abort`; DMA không phát burst mới, hoàn tất mọi burst đã cam kết (đã bật VALID hoặc đã gửi beat W), rồi kết thúc lệnh bằng `o_done` hoặc `o_error`. Sau đó FSM phát `soft_clr`
- Nếu DMA không chạy lệnh (IDLE, CHECK_CFG, COMPUTE): FSM phát `soft_clr` ở chu kỳ kế tiếp; COMPUTE đang chạy bị hủy
- Trong cả chuỗi soft reset: BUSY = 1; START và SOFT_RESET ghi lặp lại bị bỏ qua
- Sau `soft_clr`: STATUS = 0; tác vụ bị hủy không báo DONE cũng không báo ERROR; vùng đích trong DRAM không xác định
- Nếu yêu cầu soft reset đến đúng chu kỳ DMA kết thúc lệnh, coi như DMA đã rảnh và áp dụng (Nếu DMA không chạy lệnh (IDLE, CHECK_CFG, COMPUTE): FSM phát `soft_clr` ở chu kỳ kế tiếp; COMPUTE đang chạy bị hủy). Khi đã có yêu cầu soft reset, FSM không phát lệnh mới

5. Thành phần bổ sung

Hai thành phần dưới đây chỉ làm nếu còn thời gian: **DMA theo chuỗi descriptor** (CPU ra lệnh một lần cho nhiều phép GEMM) và **scratchpad ping-pong** (nạp, tính, ghi của các phép GEMM kế nhau chạy chồng lên nhau). Chúng đã được đặc tả đầy đủ ở đây và dùng các offset, bit đã dành sẵn, nên khi hiện thực không phải sửa các mục 1 – 6.

5.1. Nguyên tắc chung

Nguyên tắc của phần bổ sung

- Không thêm, bớt hay đổi port top-level (mục 1.5); không đổi offset hay bit của register map. Chỉ dùng CTRL[2], STATUS[3], các offset 0x40 … 0x48 và bit `HW_CONFIG[17:16]`
- Bật bằng tham số `HAS_CHAIN`, `HAS_PINGPONG`; phần cứng báo qua `HW_CONFIG`. Firmware phải đọc `HW_CONFIG` trước khi dùng phần bổ sung
- Tác vụ START (chế độ đơn) giữ nguyên mọi hành vi của mục 1 – 6, dù phần bổ sung có hay không

- Phần bổ sung chỉ thay đổi port nội bộ: DMA tách thành kênh đọc và kênh ghi chạy độc lập, thêm lệnh FETCH; các cổng SRAM thêm bit chọn bank; FSM thêm bộ lập lịch và hàng đợi descriptor

5.2. DMA theo chuỗi descriptor

CPU dựng sẵn một danh sách descriptor trong DRAM, mỗi descriptor mô tả một phép GEMM. CPU ghi địa chỉ descriptor đầu vào DESC_PTR, thực hiện `fencerw,rw`, ghi CTRL[2] = 1 (START_CHAIN), rồi chờ DONE một lần cho cả chuỗi. Accelerator tự đọc từng descriptor và chạy hết chuỗi.

Bảng 29: Bố cục descriptor (16 word, 64 byte)

Offset	Word	Tên	Bit	Nội dung
0x00	0	DIM	[31:0]	M, N, K, bố cục như MATRIX_DIM
0x04	1	A_ADDR	[31:0]	Địa chỉ byte của A[0][0]
0x08	2	B_ADDR	[31:0]	Địa chỉ byte của B[0][0]; bỏ qua khi REUSE_B = 1
0x0C	3	C_ADDR	[31:0]	Địa chỉ byte của C[0][0] hoặc Y[0][0]
0x10	4	LDA, LDB	[15:0], [31:16]	Stride theo hàng của A và B (byte); 0 = các hàng liền nhau
0x14	5	LDC	[15:0]	Stride của C hoặc Y (byte); 0 = liền nhau; [31:16] dự trữ
0x18	6	FLAGS	[4:0]	[0] LAST, [1] WAIT_PREV, [2] REUSE_B, [3] POST_EN, [4] RELU_EN; [31:5] dự trữ
0x1C	7	NEXT_PTR	[31:0]	Địa chỉ descriptor kế tiếp, căn 64 byte; bỏ qua khi LAST = 1
0x20	8	BIAS_ADDR	[31:0]	Như thanh ghi BIAS_ADDR; chỉ dùng khi POST_EN = 1
0x24	9	QM0	[31:0]	Như QUANT_M0; chỉ dùng khi POST_EN = 1
0x28	10	QSHIFT	[4:0]	Như QUANT_SHIFT; [31:5] dự trữ
0x2C	11	TAG	[31:0]	Tùy phần mềm; phần cứng không đọc
0x30–0x3C	12–15	—	—	Dự trữ, phải bằng 0

Với stride, địa chỉ phần tử là: A[i][k] tại A_ADDR + i·LDA + k; B[k][j] tại B_ADDR + k·LDB + j; C[i][j] tại C_ADDR + i·LDC + 4j; Y[i][j] tại C_ADDR + i·LDC + j. Stride = 0 được hiểu là đúng độ dài hàng.

Chế độ chuỗi

- DESC_PTR và mọi NEXT_PTR được dùng phải căn 64 byte; nếu không, ERR_CODE = 1. Mỗi descriptor được đọc đúng một lần, theo thứ tự chuỗi, bằng một burst INCR 16 beat (lệnh DMA FETCH)
- Mỗi descriptor chạy các pha FETCH → kiểm tra → LOAD_BIAS (nếu POST_EN) → LOAD_A → LOAD_B (trừ khi REUSE_B) → COMPUTE → STORE_C, với nội dung giống hệt một tác vụ chế độ đơn có cùng tham số
- Descriptor hợp lệ khi: M, N, K đúng miền; DIM[31:24] = 0; mọi địa chỉ được dùng căn 4 byte; mỗi stride bằng 0 hoặc ≥ độ dài hàng (K cho A; N cho B; 4N cho C; N cho Y) và là bội của 4; bit và word dự trữ bằng 0; RELU_EN kéo theo POST_EN; REUSE_B chỉ khi không phải descriptor đầu và K, N bằng của descriptor trước. Sai → ERR_CODE = 1
- REUSE_B = 1: không nạp B; COMPUTE dùng B của descriptor ngay trước
- WAIT_PREV = 1: các pha LOAD của descriptor chỉ bắt đầu khi STORE_C của mọi descriptor trước nó đã xong. Phần mềm phải đặt cờ này khi descriptor đọc dữ liệu do descriptor trước ghi ra
- Chuỗi kết thúc ở descriptor đầu tiên có LAST = 1. DONE = 1 khi STORE_C của descriptor đó nhận phản hồi B cuối là OKAY
- DESC_DONE_CNT tăng 1 mỗi khi STORE_C của một descriptor xong OKAY; các descriptor xong đúng theo thứ tự chuỗi
- Lỗi: gọi f là descriptor có thứ tự nhỏ nhất bị lỗi. Không pha nào của descriptor thứ ≥ f được bắt đầu; các descriptor đứng trước f được chạy xong. Khi mọi khối đã rảnh: ERROR = 1, ERR_CODE theo bảng mã lỗi (lỗi FETCH là mã 4), ERR_DESC_ADDR = địa chỉ của f, DESC_DONE_CNT = số thứ tự của f

- Khi dùng stride lớn hơn độ dài hàng, DMA cắt burst theo từng hàng: burst không bao giờ trải qua khoảng hở giữa hai hàng, và các byte trong khoảng hở không bị đọc hay ghi
- Accelerator không ghi vào descriptor; phần mềm không sửa descriptor và vùng dữ liệu của chuỗi đang chạy. Vùng đích của mọi descriptor không được chồng lên vùng descriptor
- Ở chế độ chuỗi, tham số VPU lấy từ descriptor; các thanh ghi `MATRIX_DIM`, `BASE_ADDR_*`, `POST_CTRL`, `BIAS_ADDR`, `QUANT_*` không được dùng

5.3. Scratchpad ping-pong

Mỗi buffer A, B, C, BIAS có hai bank. DMA kênh đọc nạp descriptor d+1 vào một bank trong lúc array tính descriptor d trên bank kia, và DMA kênh ghi chép kết quả của descriptor d−1 ra DRAM. Thời gian mỗi descriptor tiến tới max(nạp, tính, ghi) thay vì tổng của chúng. Phần này đòi hỏi chế độ chuỗi.

Bảng 30: Chọn bank cho descriptor d

Buffer	Quy tắc
A, C	Đảo bank sau mỗi descriptor
B	Đảo sau mỗi lần có LOAD_B; với `REUSE_B` = 1, dùng lại bank của descriptor trước
BIAS	Đảo sau mỗi descriptor có `POST_EN` = 1

Ping-pong

- Ba khối thực thi: Đọc (FETCH, kiểm tra, LOAD_*; kênh đọc DMA), Tính (COMPUTE; array và VPU), Ghi (STORE_C; kênh ghi DMA). Mỗi khối làm một việc một lúc, theo thứ tự descriptor. Kênh đọc và kênh ghi được phép có burst AXI cùng lúc, mỗi kênh tối đa 1 burst outstanding
- Khối Đọc làm lần lượt FETCH(d), kiểm tra d, các LOAD của d, rồi mới sang FETCH(d+1)
- LOAD vào một bank chỉ bắt đầu khi COMPUTE cuối cùng đọc bank đó đã xong
- COMPUTE(d) bắt đầu khi mọi LOAD của d và COMPUTE(d−1) đã xong, và STORE_C cuối cùng đọc bank C của d đã xong
- STORE_C(d) bắt đầu khi COMPUTE(d) và STORE_C(d−1) đã xong; do đó kết quả ra DRAM đúng theo thứ tự descriptor
- `WAIT_PREV` chặn các LOAD, không chặn FETCH và kiểm tra
- Tại mỗi thời điểm, mỗi bank có tối đa một bên truy cập. Hai bank khác nhau của cùng buffer được truy cập độc lập trong cùng chu kỳ; mỗi bank vẫn chỉ cần SRAM một cổng
- Các điều kiện thứ tự và quyền truy cập bank nêu trên là điều kiện cần; phần cứng được phép bắt đầu một pha muộn hơn, không bao giờ sớm hơn. Nội dung DRAM sau tác vụ phải giống hệt khi chạy không ping-pong
- Soft reset: abort cả hai kênh DMA đang bận; `soft_clr` phát khi cả hai kênh đã rảnh
- `PERF_LOAD` + `PERF_COMPUTE` + `PERF_STORE` được phép lớn hơn `PERF_TOTAL`; phần chênh là thời gian chạy chồng

6. Tích hợp các module

Bản chụp thanh ghi chỉ đi vào Main FSM. FSM cấp cấu hình của job đang tính cho array/VPU; cấp bank cho SRAM; cấp đầy đủ địa chỉ, số hàng, số word, stride và bank trong từng lệnh DMA. Cách nối này dùng cho cả cấu hình cơ bản và đầy đủ; khi tắt chuỗi/ping-pong, FSM tuần tự hóa các pha và dùng bank 0.

Bảng 31: Kết nối giữa các module (bản full option)

Từ	Tới	Tín hiệu
RF `o_start_pulse`, `o_start_kind`, `o_soft_req`, `o_cfg_*`	FSM `i_*` cùng tên	Xung start/soft và bản chụp cấu hình

Bảng 31: Kết nối giữa các module (bản full option) (tiếp theo)

Từ	Tới	Tín hiệu
FSM `o_done_pulse`, `o_err_*`, `o_perf_*`, `o_soft_busy`, `o_err_desc_addr`, `o_desc_done_cnt`	RF `i_*` cùng tên	
FSM `o_soft_clr`	`i_soft_clr` của RF, DMA, SRAM, Array, VPU	
FSM `o_rd_*`, `o_wr_*`	DMA `i_rd_*`, `i_wr_*`	Lệnh hai kênh
DMA `o_rd_busy/done/error`, `o_desc_*`, `o_wr_busy/done/error`	FSM `i_*` cùng tên	
DMA `o_spm_wr_*`, `o_spm_rd_*`	SRAM `i_dw_*`, `i_dr_*`	Word dữ liệu, địa chỉ, bank và bắt tay valid/ready
SRAM `o_dw_ready`, `o_dr_ready`, `o_dr_rvalid`, `o_dr_rdata`	DMA `i_spm_*`	
FSM `o_pe_start`; Array `o_pe_done`, `o_busy`	Array `i_pe_start`; FSM `i_pe_done`, `i_pe_busy`	
FSM `o_comp_dim`	Array `i_matrix_dim`	
FSM `o_comp_post_en/relu_en/qm0/qshift`	VPU `i_post_en/relu_en/quant_m0/quant_shift`	
FSM `o_comp_bank_*`, `o_comp_active`	SRAM `i_comp_bank_*`, `i_comp_active`	
Array `o_a_rd_*`, `o_b_rd_*` ↔ SRAM	SRAM `i_a_rd_*`, `i_b_rd_*`, `o_a_rd_data`, `o_b_rd_data`	Đọc vector; dữ liệu trả về trễ một chu kỳ
Array `o_row_*`	VPU `i_row_*`	Luồng hàng gồm valid, data, idx, col và mask; không ready
VPU `o_bias_rd_*`, `o_c_wr_*` ↔ SRAM	SRAM `i_bias_rd_*`, `o_bias_rd_data`, `i_c_wr_*`	
top `s_axi_*`	RF	17 tín hiệu
DMA `m_axi_*`	top	25 tín hiệu

Tham số P phải thống nhất giữa array, VPU, SRAM và cấu hình phần cứng báo cho phần mềm. Đặt L_VPU của array bằng LAT = 3 + MUL_STAGES của VPU. HAS_PINGPONG chỉ hợp lệ khi HAS_CHAIN = 1; top truyền giá trị tham số đã chọn xuống các module, không phụ thuộc mặc định riêng của từng module.